Python 基础梳理与典型题详解

变量、类型、输入输出、比较与逻辑运算、条件分支

· 10道单选 · 5 道多选 · 5 道填空 · 4 道综合题

语义基准　Python 3 当前稳定版官方文档（核验日：2026-07-25）
编排原则　原稿内容梳理、权威来源交叉核验、原创典型题、逐步解析
适用对象　Python 入门学习、自测、复习与课堂练习

使用说明

本册先把四张手写图片中的知识按原顺序重新编码，再给出基于 Python 官方语言参考、标准库文档与 PEP 的规范校正；随后提供恰好 24 道原创练习，最后给出逐项、逐式、逐分支的详细答案。题目不是从网络题库照搬，而是以公开一手资料核验过的语义为依据重新命制。

建议使用顺序

1. 先读“原稿数字化重构”，确认四页笔记覆盖的知识边界。
2. 再读“官方语义校正”，尤其注意输入类型、短路求值、链式比较和分支顺序。
3. 独立完成题目部分；多选题至少有两个正确选项。
4. 最后对照详解，不只核对答案，还要核对每一步为什么成立。

符号约定

- $a \mid b$：a 整除b；$a \nmid b$：a 不能整除b。
- $\wedge, \vee, \neg$：逻辑“且”“或”“非”。
- 代码与输出均按 Python 3 解释；除非题目另有说明，假设用户输入可完成题目要求的类型转换。

目录

使用说明 i

第一部分　原稿重构与知识校正 1

第一章　四页手写笔记的数字化重构 2

1.1　图片 1：变量、基本类型、类型查看与输入输出 2

1.2　图片 2：比较运算、逻辑运算与条件判断入口 3

1.3　图片 3：基本分支与双分支 4

1.4　图片 4：多分支、正负零与书写规则 4

第二章　经权威资料核验的核心知识 6

2.1　名称、对象与赋值 6

2.2　基本类型、`type()` 与 `isinstance()` 6

2.3　输入、转换、输出与 f-string 7

2.4　比较、链式比较与逻辑运算 7

2.4.1　真值与短路公式 7

2.5　条件分支与区间覆盖 8

2.6　公历闰年的完整公式 8

第二部分　典型练习 9

第三章　单项选择题 10

第四章　多项选择题 14

第五章　填空题 16

第六章　综合题 17

第三部分　标准答案与极详解析 19

第七章　单项选择题详解 20

7.1 第 1 题：input() 的返回类型 . . . 20
7.2 第 2 题：赋值后的变量值 . . . 20
7.3 第 3 题：不同类型数据的相等比较 . . . 21
7.4 第 4 题：f-string 输出 . . . 22
7.5 第 5 题：逻辑运算符优先级 . . . 22
7.6 第 6 题：分支边界 . . . 23
7.7 第 7 题：独立判断与 else 归属 . . . 23
7.8 第 8 题：闭区间与德摩根律 . . . 24
7.9 第 9 题：直接类型与继承关系 . . . 24
7.10 第 10 题：闰年条件 . . . 25

第八章 多项选择题详解 26

8.1 第 11 题：类型与转换 . . . 26
8.2 第 12 题：比较与逻辑表达式 . . . 26
8.3 第 13 题：条件语句的语法与执行 . . . 27
8.4 第 14 题：独立 if 的输出 . . . 27
8.5 第 15 题：闰年边界 . . . 27

第九章 填空题详解 29

9.1 第 16 题：将输入转换为整数 . . . 29
9.2 第 17 题：补全 f-string . . . 29
9.3 第 18 题：闭区间判断 . . . 29
9.4 第 19 题：补全多分支关键词 . . . 30
9.5 第 20 题：补全闰年核心条件 . . . 30

第十章 综合题详解 31

10.1 第 21 题：独立判断与分支配对 . . . 31
10.1.1 （1）当 score = 80 . . . 31
10.1.2 （2）改为 90 分与 59 分 . . . 31
10.1.3 （3）为什么 80 分同时出现 P 与 B . . . 32
10.1.4 （4）变量值汇总 . . . 32
10.2 第 22 题：成绩等级程序纠错 . . . 32
10.2.1 （1）错误定位、分类与影响 . . . 32
10.2.2 （2）完整修正版 . . . 33
10.2.3 （3）边界测试 . . . 34
10.3 第 23 题：票种分段程序 . . . 34

10.3.1 （1）完整程序 34
10.3.2 （2）为什么后续分支只写上界 34
10.3.3 （3）边界输出 35
10.3.4 （4）错误条件的后果 35
10.4 第 24 题：公历闰年 35
10.4.1 （1）数学命题 35
10.4.2 （2）完整程序 36
10.4.3 （3）逐年验证 36
10.4.4 （4）两类典型错误 37
10.4.5 400 年周期的独立验算 37

第四部分 核验附录 39

附录 A 答案速查与难度分布 40
A.1 客观题答案速查 40
A.2 题量与难度审计 40
A.3 知识覆盖 40

附录 B 资料来源与全球检索核验 42
B.1 Python 官方语言与标准库资料 42
B.2 闰年规则的交叉来源 42
B.3 核验结论摘要 42

附录 C 最终自检清单 44

第一部分

原稿重构与知识校正

第一章　四页手写笔记的数字化重构

重构说明

以下内容按四张原图的知识顺序重排。原稿中因拍摄角度、笔迹或初学阶段表述造成的歧义，会保留其知识意图，但不会把明显不完整或不可执行的代码冒充成正确代码；紧随其后的“规范校正”将逐条给出准确语义。

1.1　图片 1：变量、基本类型、类型查看与输入输出

原稿内容重写

第一章：核心语法

一、变量

1. 变量：程序中用来保存数据的名称，可在输出、计算与记录数据时使用。
2. 定义变量的基本格式：变量名 = 表达式的值。
3. 使用变量：参与输出、参与计算、记录程序运行中的数据。
4. 原稿提醒：变量先赋值后使用；一条语句可连续赋值，也可同时为多个名称赋值。

二、常见数据类型

int	float	str	bool	NoneType
整数	浮点数	字符串	布尔值	空值类型

三、查看类型

- type(数据)：查看对象的直接类型。
- isinstance(数据，类型)：判断对象是否属于指定类型或其子类，返回布尔值。

四、输入与输出

```
name = input("请输入您的姓名：")
age = input("请输入您的年龄：")
print(f"您的姓名为{name}，年龄为{age}")
```

规范校正

“变量是容器”便于入门理解，但更准确的说法是：**名称绑定到对象**。重新给一个名称赋值，是让它重新绑定到另一个对象，不会自动修改别的名称。另一个关键点是：`input()` 的返回值先是 `str`，即使用户输入的字符看起来像数字。[S1][S2][S3]

1.2　图片 2：比较运算、逻辑运算与条件判断入口

原稿内容重写

一、输入提醒

`input()` 输入的数据类型均为字符串；若后续需要数值比较或计算，应进行显式类型转换。

二、比较运算符

运算符	名称	示例含义
`==`	等于	`a == b` 判断两个值是否相等
`!=`	不等于	`a != b` 判断两个值是否不相等
`>`	大于	`a > b`
`>=`	大于等于	`a >= b`
`<`	小于	`a < b`
`<=`	小于等于	`a <= b`

三、逻辑运算符

运算符	名称	原稿意图
`and`	逻辑与	条件同时满足
`or`	逻辑或	条件至少一个满足
`not`	逻辑非	对真值取反

四、区间示例

```
num = int(input("请输入一个整数："))
print(f"输入整数 {num} 在 10~20 之间：{num >= 10 and num <= 20}")
```

规范校正

`and` 与 `or` 不只是把结果变成 `True/False`：它们会短路，并返回最后实际求值的操作数。`not` 才总返回布尔值。闭区间还可写成链式比较 `10 <= num <= 20`。[S4][S5]

1.3　图片 3：基本分支与双分支

原稿内容重写

一、基本 if 结构

```
if 条件:
    条件成立时执行的语句
```

条件表达式会进行真值测试。语句头末尾要有英文冒号，分支体需要缩进。

二、if--else 双分支

```
if 条件:
    条件成立时执行操作 1
else:
    条件不成立时执行操作 2
```

三、登录与闰年示例的知识意图

- 登录：账号与密码条件同时成立时输出“登录成功”，否则输出“登录失败”。
- 闰年：通过余数、比较与逻辑运算组合判断年份。

规范校正

if 的条件不必“类型恰好为 bool”；任何对象都可进行真值测试。空字符串、数值零、None 等是假值。账号与密码验证应使用 == 做值比较，而不是用 = 或 is。[S4][S6]

1.4　图片 4：多分支、正负零与书写规则

原稿内容重写

一、else 的注意事项

- else 不需要条件；前面的 if 条件不成立时才执行。
- else: 后的代码块也必须缩进。

二、if--elif--else 多分支

```
num = int(input("请输入数字: "))

if num < 0:
    print("该数字为负数")
elif num > 0:
    print("该数字为正数")
else:
    print("该数字为 0")
```

三、原稿提醒

- `elif` 可以有多个。
- `else` 至多一个，若存在应放在整条分支链的最后。
- 多个条件按书写顺序判断。

规范校正

同一条 `if--elif--else` 链会从上到下寻找第一个真条件，并且**至多执行一个分支**；多个彼此独立的 `if` 则可能同时执行。分支顺序会影响区间覆盖，通常先处理非法输入或更特殊的情况。[S6][S7]

第二章　经权威资料核验的核心知识

2.1　名称、对象与赋值

Python中，每个对象都有标识、类型和值。赋值语句把名称绑定或重新绑定到对象；右侧表达式先求值，所得对象再绑定到左侧目标。[S1][S2]

```
a = 3
b = a
a = 8
print(a, b)   # 8 3
```

执行轨迹为

语句执行后	a	b
$a = 3$	3	未定义
$b = a$	3	3
$a = 8$	8	3

最后一次赋值只重新绑定a，不会让b 自动跟随变化。

2.2　基本类型、type() 与 isinstance()

类型	示例	关键性质
int	42, -7	任意精度整数
float	3.14	浮点数，许多十进制小数只能近似表示
str	"Python"	Unicode 文本，不可变序列
bool	True, False	只有两个实例；是 int 的子类
NoneType	None	None 的类型；None 是单例

```
x = True
print(type(x) is bool)       # True: 直接类型是 bool
print(type(x) is int)        # False: 不是“恰好为 int”
print(isinstance(x, int))    # True: bool 是 int 的子类
```

易错点

type(x) is T 检查直接类型；isinstance(x, T) 会考虑继承关系。两者在子类场景中不等价。普通值相等应使用 ==；is 判断是否为同一对象。[S1][S3][S4]

2.3 输入、转换、输出与 f-string

`input(prompt)` 先显示提示，再读取一行，去掉行末换行符，返回 `str`。[S3]

```
raw = input("请输入年龄: ")  # 输入 18 时，raw 是字符串 "18"
age = int(raw)              # 显式转换为整数 18
print("年龄", age, sep=": ", end="\n")
```

```
print(*objects, sep=' ', end='\n', file=None, flush=False)
```

它会把对象转换为文本，以 `sep` 分隔，并在末尾写出 `end`。

f-string 以 `f` 或 `F` 为前缀，花括号内的表达式在运行时从左到右求值：

```
name = "Lin"
score = 92
print(f"{name}:{score}")       # Lin:92
print(f"{score / 100:.1%}")    # 92.0%
```

其中 `.1%` 是显示格式，表示百分比并保留一位小数；它不改变变量 `score` 的原始值。Python 3.12 起，PEP 701 移除了若干早期 f-string 表达式限制，因此不能把 PEP 498 的全部旧限制机械套用于当前版本。[S8][S9]

2.4 比较、链式比较与逻辑运算

比较运算通常得到布尔值。链式比较

$$10 \le n \le 20$$

在条件意义上等价于

$$(10 \le n) \wedge (n \le 20),$$

并保证中间表达式n 只求值一次；若第一段为假，后段不会求值。[S5]

2.4.1 真值与短路公式

$$x \text{ and } y = \begin{cases} x, & x \text{ 为假值}, \\ y, & x \text{ 为真值}, \end{cases} \qquad x \text{ or } y = \begin{cases} x, & x \text{ 为真值}, \\ y, & x \text{ 为假值}. \end{cases}$$

因此：

```
0 and 1 / 0       # 0；右侧未求值，所以没有除零异常
"" or "default"   # "default"
"A" and 7         # 7
not []            # True；not 总返回 bool
```

优先级从高到低为

$$\text{比较运算} \ > \ \texttt{not} \ > \ \texttt{and} \ > \ \texttt{or}.$$

复杂表达式即使可依赖优先级，也应通过括号明确意图。[S4][S5]

2.5　条件分支与区间覆盖

```
if condition_1:
    suite_1
elif condition_2:
    suite_2
else:
    suite_3
```

条件按书写顺序求值。遇到第一个真条件后，执行对应代码组，其余条件与代码组都不再求值；若全部为假才执行可选的 `else`。[S6]

缩进是语法的一部分。建议每级 4 个空格；不要以含义依赖制表宽度的方式混用制表符和空格，否则可能触发 `TabError`。[S7]

2.6　公历闰年的完整公式

设年份为整数y。公历闰年命题为

$$\boxed{L(y) = (4 \mid y) \wedge \left[(100 \nmid y) \vee (400 \mid y)\right]}$$

等价地，

$$\boxed{L(y) = \big((4 \mid y) \wedge (100 \nmid y)\big) \vee (400 \mid y)}.$$

Python表达式为：

```
is_leap = year % 4 == 0 and (
    year % 100 != 0 or year % 400 == 0
)
```

400 年周期中：

$$\frac{400}{4} - \frac{400}{100} + \frac{400}{400} = 100 - 4 + 1 = 97$$

个闰年。因此总天数为

$$303 \times 365 + 97 \times 366 = 146097,$$

平均历年长度为

$$\frac{146097}{400} = 365.2425 \text{ 日}.$$

1900、2100 是非闰世纪年；2000、2400 能被 400 整除，是闰年。[S11][S12][S13]

第二部分

典型练习

第三章　单项选择题

1. input() 的返回类型

考点：输入与字符串　|　难度：基础

执行下列语句，用户在键盘上输入 18 并按回车：

```
age = input("请输入年龄：")
```

此时变量 age 的数据类型是（　）。

A. int

B. float

C. str

D. bool

2. 赋值后的变量值

考点：变量与赋值　|　难度：基础

下列程序的输出是（　）。

```
a = 3
b = a
a = 8
print(a, b)
```

A. 3 3

B. 8 8

C. 3 8

D. 8 3

3. 不同类型数据的相等比较

考点：str、int 与 ==　|　难度：基础

```
x = "5"
y = 5
print(x == y)
```

程序输出是（　）。

A. `True`

B. `False`

C. `5`

D. 程序抛出 `TypeError`

4. f-string 输出

考点：输出与格式化字符串　|　难度：基础

```
name = "Lin"
score = 92
print(f"{name}:{score}")
```

程序输出是（　）。

A. `{name}:{score}`

B. `Lin:92`

C. `"Lin:92"`

D. `fLin:92`

5. 逻辑运算符优先级

考点：and、or 与优先级　|　难度：中等

```
p = True
q = False
r = False
print(p or q and r)
```

程序输出是（　）。

A. `True`

B. `False`

C. `None`

D. 程序存在语法错误

6. 分支边界

考点：严格大于与大于等于　|　难度：中等

```
score = 60
if score > 60:
    print("通过")
else:
    print("需努力")
```

程序输出是（　）。

A. 通过

B. 需努力

C. 两行都输出

D. 什么也不输出

7. 两个独立 `if` 与 `else` 的归属

考点：独立判断、缩进与分支配对　|　难度：中等

```
n = 0
if n >= 0:
    print("A")
if n == 0:
    print("B")
else:
    print("C")
```

程序输出是（　）。

A. 只输出 `A`

B. 依次输出 `A`、`B`

C. 依次输出 `A`、`C`

D. 只输出 `B`

8. 闭区间条件的等价表达

考点：逻辑等价、边界与德摩根律　|　难度：中等

若要判断整数n 是否位于闭区间$[10, 20]$ 内，即同时允许 10 和 20，下列表达式中唯一正确的是（　）。

A. `n > 10 and n < 20`

B. `n >= 10 or n <= 20`

C. `not (n < 10 or n > 20)`

D. `not (n >= 10 and n <= 20)`

9. type() 与 isinstance() 的细节

考点：布尔类型、直接类型与继承关系　|　难度：提升

```
x = True
print(type(x) == bool, isinstance(x, int))
```

程序输出是（　）。

A. `True True`

B. `True False`

C. `False True`

D. `False False`

10. 闰年条件

考点：余数、比较与逻辑组合　|　难度：提升

设 `year` 是正整数。下列哪个条件能准确判断公历闰年（　）？

A. `(year % 4 == 0 and year % 100 != 0) or year % 400 == 0`

B. `year % 4 == 0`

C. `year % 100 == 0 or year % 400 == 0`

D. `year % 4 == 0 and year % 400 == 0`

第四章　多项选择题

每题至少有两个正确选项。少选、错选或多选都会暴露不同的概念缺口；详解将逐项判断。

11. 类型与转换

考点：基本类型、输入与显式转换　|　难度：中等

下列说法正确的是（　）。

A. `input()` 读入的内容默认是字符串

B. `int("12")` 的结果是整数 12

C. `float("3.5")` 的结果是浮点数 3.5

D. `bool("False")` 的结果是布尔值 `False`

E. `None` 的类型名称是 `NoneType`

12. 比较与逻辑表达式

考点：比较、and/or/not　|　难度：基础

已知 `n = 10`，下列表达式结果为 `True` 的是（　）。

A. `n >= 10`

B. `n > 10`

C. `n == 10 and n < 20`

D. `n != 10 or n < 0`

E. `not (n < 10)`

13. 条件语句的语法与执行

考点：if/elif/else　|　难度：中等

下列说法正确的是（　）。

A. `if`、`elif`、`else` 的语句头末尾都要写英文冒号

B. `elif` 可以脱离 `if` 单独使用

C. 同一条分支链中至多执行一个分支

D. 多个彼此独立的 `if` 条件可以在同一次运行中都成立

E. `else` 后面应直接书写一个条件表达式

14. 独立 `if` 的输出

考点：分支结构、`else` 归属与执行次数　|　难度：中等

```
x = 75
if x >= 60:
    print("P")
if x >= 80:
    print("G")
else:
    print("N")
```

下列说法正确的是（　）。

A. 当前程序第一行输出为 `P`

B. 当前程序第二行输出为 `N`

C. 当前程序会输出 `G`

D. `else` 与第一个 `if x >= 60` 配对

E. 若把 `x` 改为 85，程序依次输出 `P`、`G`，且不输出 `N`

15. 判断哪些年份是闰年

考点：公历闰年与世纪年边界　|　难度：中等

按照公历规则，下列年份中属于闰年的是（　）。

A. 1896

B. 1900

C. 2000

D. 2100

E. 2024

第五章　填空题

16. 将输入转换为整数

考点：input() 与 int()　|　难度：基础

补全代码，使用户输入合法的整数字符串后，age 保存整数值。

```
age = ______(input("请输入年龄："))
```

17. 补全 f-string

考点：格式化输出　|　难度：基础

```
print(______"姓名：{name}，年龄：{age}")
```

18. 闭区间判断

考点：比较与 and　|　难度：中等

只使用比较运算符和 and，写出“n 位于闭区间$[-5,5]$ 内”的完整条件表达式。

19. 补全多分支关键词

考点：if--elif--else　|　难度：中等

```
if score >= 90:
    print("优秀")
______ score >= 60:
    print("合格")
______:
    print("待提高")
```

20. 补全闰年核心条件

考点：逻辑组合与余数　|　难度：提升

补全两个比较运算符，使表达式准确表示公历闰年。

```
(year % 4 == 0 and year % 100 ______ 0) or year % 400 ______ 0
```

第六章　综合题

21. 代码阅读：独立判断与分支配对

考点：执行顺序、比较、布尔值与 f-string ｜ 难度：中等

```
score = 80
passed = score >= 60
excellent = score >= 90

if passed:
    print("P")

if excellent:
    print("A")
else:
    print("B")

print(f"{score}:{passed}")
```

1. 写出 `score = 80` 时的全部输出，注意行序。
2. 分别把第一行改为 `score = 90` 和 `score = 59`，其余代码不变，各写出全部输出。
3. 解释为什么 80 分时既输出 `P` 又输出 `B`；指出 `else` 与哪一个 `if` 配对。
4. 写出三个输入值下 `passed` 与 `excellent` 的布尔值。

22. 程序纠错：成绩等级

考点：输入类型、比较、冒号、缩进、边界与输出 ｜ 难度：提升

程序原意是：输入整数成绩；若不在0 到100 之间，输出“输入无效”；否则，90–100 为 A，60–89 为 P，0–59 为 F。

```
score = input("请输入0~100之间的整数成绩：")
if score < 0 or score > 100:
    print("输入无效")
elif score = 90:
    grade = "A"
elif score >= 60
grade = "P"
else score < 60:
```

```
    grade = "F"
print("等级：{grade}")
```

1. 找出并解释错误，说明它们属于类型、语法、缩进、边界还是输出格式问题。
2. 写出完整、可运行且所有路径都有确定输出的修正版。
3. 给出输入$-1, 59, 60, 89, 90, 100, 101$ 时修正版的输出。

23. 分段程序设计：票种判定

考点：分段条件、边界覆盖与多分支 | 难度：中等

某场馆按整数年龄a 判定票种：

$$T(a)=\begin{cases}\text{输入无效}, & a<0,\\ \text{免票}, & 0\le a\le 5,\\ \text{未成年人票}, & 6\le a\le 17,\\ \text{成人票}, & 18\le a\le 64,\\ \text{长者票}, & a\ge 65.\end{cases}$$

1. 使用 `input()`、`int()`、`if--elif--else` 与 f-string 编写完整程序，输出格式为“票种：判定结果”。
2. 解释为何后续 `elif` 中只检查上界仍能保证区间正确。
3. 写出年龄$-1, 0, 5, 6, 17, 18, 64, 65$ 的输出。
4. 若错误地把第二个分支写成 `elif age < 5:`，会漏判哪个边界值？

24. 综合应用：公历闰年

考点：余数、逻辑等价、分支、边界与验证 | 难度：提升

设 `year` 为正整数。公历规则为：能被 400 整除，或能被 4 整除但不能被 100 整除的年份为闰年。

1. 用数学符号写出闰年命题$L(y)$。
2. 编写完整程序：输入年份；若转换后的整数值小于等于 0，输出“年份必须为正整数”；否则输出“年份是闰年”或“年份不是闰年”，并用 f-string 插入年份。
3. 分别判断 1896、1900、2000、2023、2024、2100，并逐项说明理由。
4. 解释为什么只写 `year % 4 == 0` 不够，为什么先判断整百年时也必须谨慎处理 400 年例外。

第三部分

标准答案与极详解析

第七章　单项选择题详解

7.1　第 1 题：input() 的返回类型

标准答案

C

逐步推理

1. input(" 请输入年龄：") 先把提示文字写到标准输出，默认不在提示后额外换行。
2. 用户键入的是字符 1、8，回车表示这一行输入结束。
3. input() 去掉行末换行符，把其余内容组成字符串并返回。
4. 所以 age 的值是字符串"18"，类型为 str。

逐项判断

- A 错：只有显式写成 int(input(...))，且输入文本可转为整数时，最终结果才是 int。
- B 错：需要显式使用 float(...) 才可能得到 float。
- C 对：这是 input() 的标准返回类型。
- D 错：input() 不会把文本自动变成布尔值。

易错点

输入 18、-3、3.14、True，甚至直接回车，返回值都仍是字符串；直接回车得到空字符串""。转换失败时，如执行 int("3.5")，会抛出 ValueError。[S3]

7.2　第 2 题：赋值后的变量值

标准答案

D

执行状态如下：

执行语句	a	b
$a = 3$	3	未定义
$b = a$	3	3
$a = 8$	8	3

`b = a` 执行时，名称 `b` 绑定到当时 `a` 所表示的整数对象；随后 `a = 8` 只让 `a` 重新绑定，不会自动改变 `b`。`print(a, b)` 默认以一个空格分隔两个对象，因此输出

```
8 3
```

逐项判断

- A 忽略了最后一次对 `a` 的赋值。
- B 错把 `b` 理解为会自动跟随 `a` 的“动态别名”。
- C 颠倒了输出时两个名称的当前绑定。
- D 与完整执行轨迹一致。

易错点

本题使用不可变整数。若两个名称绑定到同一个可变列表，原地修改列表时两个名称都能观察到变化；但“重新给其中一个名称赋值”仍只是重新绑定。这正是“名称绑定”比“变量盒子”更准确的原因。[S1][S2]

7.3 第 3 题：不同类型数据的相等比较

标准答案

B

`x` 是字符串`"5"`，`y` 是整数5。两者虽然书写上都出现字符 5，但对象的类型和值语义不同，因此

$$\texttt{"5"} == 5 = \texttt{False}.$$

- A 错：视觉相似不等于 Python 值相等。
- B 对：`==` 返回布尔值 `False`。
- C 错：比较表达式不会返回被比较的数字。
- D 错：跨类型的相等比较在这里合法并返回 `False`；但某些跨类型的次序比较，如`"5" < 5`，会抛出 `TypeError`。

若先显式转换：

```python
print(int(x) == y)   # True
```

7.4 第 4 题：f-string 输出

标准答案

B

字符串前的 `f` 启用替换字段：

$$\texttt{\{name\}} \longrightarrow \texttt{Lin}, \qquad \texttt{\{score\}} \longrightarrow 92.$$

冒号位于替换字段之外，是普通文本，原样保留。因此输出为 `Lin:92`。

- A 是缺少 `f` 前缀时容易得到的错误结果。
- B 正确。
- C 错：源码中的引号用于界定字符串，不是字符串内容。
- D 错：`f` 是语法前缀，不会输出。

易错点

要显示字面花括号，应在 f-string 的替换字段外写双花括号：`{{` 显示 `{`，`}}` 显示`}`。当前 Python 3.12+ 的 f-string 语法以 PEP 701 后的语言参考为准。[S8][S9]

7.5 第 5 题：逻辑运算符优先级

标准答案

A

逻辑运算符优先级为

$$\texttt{not} > \texttt{and} > \texttt{or}.$$

所以

$$\texttt{p or q and r} \equiv \texttt{p or (q and r)}.$$

先计算

$$\texttt{q and r} = \texttt{False and False} = \texttt{False},$$

再计算

$$\texttt{True or False} = \texttt{True}.$$

从短路角度看还可更快：`or` 左侧 `p` 已是真值，因此整个 `or` 表达式直接返回 `p`，右侧 `q and r` 在实际运行时不会求值。无论采用优先级分组还是短路追踪，最终都输出 `True`。

易错点

`and/or` 对非布尔操作数会返回某个原操作数，不保证返回类型为 `bool`。本题三个操作数都恰好是布尔值，故结果也是布尔值。[S4][S5]

7.6　第 6 题：分支边界

标准答案

B

条件为 `score > 60`。代入边界值：

$$60 > 60$$

为假，因此跳过 `if` 代码组，执行 `else`，输出“需努力”。

- A 错：严格大于号不包含相等边界。
- B 对。
- C 错：一条 `if--else` 只执行其中一个分支。
- D 错：条件为假时有明确的 `else` 分支。

若规则是“60 分及以上通过”，应写 `score >= 60`。最小边界测试组是$59, 60, 61$。

7.7　第 7 题：独立判断与 `else` 归属

标准答案

B

程序不是一条三分支链，而是两个结构：

$$\underbrace{\texttt{if n >= 0}}_{\text{结构一}} \qquad \underbrace{\texttt{if n == 0 ... else ...}}_{\text{结构二}}.$$

1. $0 \geq 0$ 为真，结构一输出 `A`。
2. $0 = 0$ 为真，结构二输出 `B`。
3. `else` 与同一缩进层级、位置最近的第二个 `if` 配对；第二个条件已成立，所以不输出 `C`。

最终输出为

```
A
B
```

易错点

缩进和结构边界共同消除“悬空 `else`”歧义。`else` 不会跨过一个同级的 `if` 去寻找更早的 `if`。[S6][S7]

7.8 第 8 题：闭区间与德摩根律

标准答案

C

闭区间$[10, 20]$ 的直接条件是

$$10 \le n \le 20.$$

其补集，即“区间外”，为

$$n < 10 \lor n > 20.$$

整体取反得

$$\neg(n < 10 \lor n > 20).$$

由德摩根律，

$$\neg(A \lor B) \equiv (\neg A) \land (\neg B),$$

所以

$$\neg(n < 10 \lor n > 20) \equiv n \ge 10 \land n \le 20.$$

逐项判断

- A 表示开区间$(10, 20)$，漏掉 10 与 20。
- B 几乎恒真：当$n < 10$ 时通常有$n \le 20$；当$n > 20$ 时有$n \ge 10$。
- C 正确。
- D 是闭区间条件的否定，表示区间外。

n	9	10	20	21
正确结果	假	真	真	假

7.9 第 9 题：直接类型与继承关系

标准答案

A

`type(True)` 的直接结果是 `bool`，所以

$$\texttt{type(x) == bool} = \texttt{True}.$$

Python 中 `bool` 是 `int` 的子类，而 `isinstance()` 会考虑继承关系，所以

$$\texttt{isinstance(True, int)} = \texttt{True}.$$

因此输出：

```
True True
```

四个关键对照

表达式	结果
`type(True) is bool`	`True`
`type(True) is int`	`False`
`isinstance(True, bool)`	`True`
`isinstance(True, int)`	`True`

7.10 第 10 题：闰年条件

标准答案

A

闰年规则可写成两个互补路径：

$$\underbrace{(4 \mid y) \wedge (100 \nmid y)}_{\text{普通闰年}} \quad \vee \quad \underbrace{400 \mid y}_{\text{世纪闰年}}.$$

翻译为 Python：

```
(year % 4 == 0 and year % 100 != 0) or year % 400 == 0
```

逐项判断

- A 完整覆盖两条路径。
- B 会把 1900、2100 等非闰世纪年误判为闰年。
- C 会因为“能被 100 整除”而错误接纳 1900、2100。
- D 实际只接受 400 的倍数，会漏掉 2024 等普通闰年。

最小代表测试集是：2023（普通平年）、2024（普通闰年）、1900（非闰世纪年）、2000（闰世纪年）。

第八章　多项选择题详解

8.1　第 11 题：类型与转换

标准答案

A、B、C、E

选项	判断与理由
A	正确。`input()` 返回 `str`。
B	正确。`"12"` 是合法整数字符串，`int("12")` 得整数12。
C	正确。`"3.5"` 是合法浮点数字符串，`float("3.5")` 得浮点数3.5。
D	错误。`bool()` 对字符串检查的是“是否非空”；`"False"` 含 5 个字符，是非空字符串，所以结果为 `True`。
E	正确。`type(None).__name__` 为`"NoneType"`。

易错点

`bool("")` 为 `False`；`bool("0")` 与 `bool("False")` 都为 `True`。字符串内容看起来像“假”并不会自动被解释成布尔假。[S3][S4]

8.2　第 12 题：比较与逻辑表达式

标准答案

A、C、E

代入$n = 10$：

A: $10 \geq 10 = \texttt{True}$,

B: $10 > 10 = \texttt{False}$,

C: $(10 = 10) \wedge (10 < 20) = \texttt{True} \wedge \texttt{True} = \texttt{True}$,

D: $(10 \neq 10) \vee (10 < 0) = \texttt{False} \vee \texttt{False} = \texttt{False}$,

E: $\neg(10 < 10) = \neg\texttt{False} = \texttt{True}$.

8.3 第 13 题：条件语句的语法与执行

标准答案

A、C、D

- A 正确：三个语句头都以英文冒号结束。
- B 错误：`elif` 是“否则如果”，必须属于已有的 `if` 链。
- C 正确：同一条链按顺序寻找第一个真条件，至多执行一个分支。
- D 正确：多个独立 `if` 会分别判断，可能同时成立。
- E 错误：`else` 表示“此前条件均不成立”，正确形式只有 `else:`，后面不加条件。

8.4 第 14 题：独立 `if` 的输出

标准答案

A、B、E

当$x = 75$：

$$75 \geq 60 = \texttt{True}, \qquad 75 \geq 80 = \texttt{False}.$$

所以第一个独立 `if` 输出 `P`；第二个 `if` 不成立，执行与它配对的 `else`，输出 `N`：

```
P
N
```

- A、B 正确。
- C 错：75 未达到 80。
- D 错：`else` 与第二个 `if` 配对。
- E 正确：当$x = 85$，两个独立 `if` 都成立，输出 `P`、`G`；第二个 `if` 成立后不执行 `else`。

8.5 第 15 题：闰年边界

标准答案

A、C、E

年份	被 4 整除	被 100 整除	被 400 整除	结论
1896	是	否	否	闰年
1900	是	是	否	平年
2000	是	是	是	闰年
2100	是	是	否	平年
2024	是	否	否	闰年

因此 A、C、E 正确。题目特意同时放入普通闰年、非闰世纪年和闰世纪年，避免把“每四年一闰”误当成无例外规则。

第九章　填空题详解

9.1　第 16 题：将输入转换为整数

标准答案

int

完整代码：

```
age = int(input("请输入年龄："))
```

`input()` 先产生字符串，`int(...)` 再把合法整数字符串转换为整数。若输入 `18.5` 或 `abc`，转换会抛出 `ValueError`；题目已限定输入合法。

9.2　第 17 题：补全 f-string

标准答案

f

```
print(f"姓名：{name}，年龄：{age}")
```

`f` 必须直接位于起始引号前；它是字符串前缀，不是输出字符。也可使用大写 `F`，但标准答案采用最常见的小写形式。

9.3　第 18 题：闭区间判断

标准答案

n >= -5 and n <= 5

闭区间的数学定义为

$$n \in [-5, 5] \iff n \geq -5 \land n \leq 5.$$

因此还可等价写成 `-5 <= n and n <= 5`。Python 支持链式比较 `-5 <= n <= 5`，但题目明确要求使用 `and`，故链式写法不作为本题标准填法。

易错点

`n > -5 and n < 5` 会漏掉两个端点；`n >= -5 or n <= 5` 对所有实数都为真，不能表示有限区间。

9.4 第 19 题：补全多分支关键词

标准答案

elif; else

```
if score >= 90:
    print("优秀")
elif score >= 60:
    print("合格")
else:
    print("待提高")
```

到达第二个条件时，已经知道第一条件为假，即score <90；再结合score ≥ 60，第二分支实际覆盖

$$60 \leq \texttt{score} < 90.$$

到达 `else` 时前两条件均为假，因此覆盖score <60。

9.5 第 20 题：补全闰年核心条件

标准答案

!=; ==

```
(year % 4 == 0 and year % 100 != 0) or year % 400 == 0
```

各部分逐字翻译为：

$$\texttt{year \% 4 == 0} \iff 4 \mid y,$$

$$\texttt{year \% 100 != 0} \iff 100 \nmid y,$$

$$\texttt{year \% 400 == 0} \iff 400 \mid y.$$

1900 应判假而 2000 应判真，是检查两个空是否填对的关键测试对。

第十章　综合题详解

10.1　第 21 题：独立判断与分支配对

10.1.1　(1) 当 score = 80

先求两个比较表达式：

$$\texttt{passed} = (80 \geq 60) = \texttt{True},$$

$$\texttt{excellent} = (80 \geq 90) = \texttt{False}.$$

逐句执行：

1. `if passed:` 的条件为真，输出 `P`。
2. 第二个 `if excellent:` 的条件为假，不输出 `A`。
3. `else` 与第二个 `if` 配对，因此输出 `B`。
4. f-string 插入 `score` 的值 80 和 `passed` 的值 `True`。

完整输出：

```
P
B
80:True
```

10.1.2　(2) 改为 90 分与 59 分

90分

$$\texttt{passed} = (90 \geq 60) = \texttt{True}, \qquad \texttt{excellent} = (90 \geq 90) = \texttt{True}.$$

两个独立 `if` 均成立；第二个 `if` 成立后不执行其 `else`：

```
P
A
90:True
```

59 分

$$\texttt{passed} = (59 \geq 60) = \texttt{False}, \qquad \texttt{excellent} = (59 \geq 90) = \texttt{False}.$$

第一个 `if` 不输出；第二个 `if` 不成立，执行其 `else`：

```
B
59:False
```

10.1.3　(3) 为什么 80 分同时出现 P 与 B

代码由两个彼此独立的结构组成：

结构一：`if passed`　　结构二：`if excellent ... else ...`.

`P` 表示“已达到 60 分”；`B` 表示“没有达到 90 分”。两条判断可以同时给出信息，逻辑上并不冲突。`else` 与同一缩进层级且位置最近的 `if excellent:` 配对。

10.1.4　(4) 变量值汇总

score	passed = score >= 60	excellent = score >= 90
80	True	False
90	True	True
59	False	False

易错点

60 和 90 都是闭边界：60 已通过，90 已优秀。不要把两个独立 `if` 误读成 `if--elif`；独立 `if` 可能同时执行，同一条分支链才至多执行一个分支。

10.2　第 22 题：成绩等级程序纠错

10.2.1　(1) 错误定位、分类与影响

序号	类别	错误及后果
1	类型错误	`input()` 返回字符串；原代码马上把字符串与整数 0、100 做次序比较，Python 3 会抛出 `TypeError`。应先用 `int()` 转换。
2	语法错误	`elif score = 90:` 中的 `=` 是赋值符号，不能充当条件中的相等比较。
3	边界错误	即使把上一处机械改成 `score == 90`，也只接收 90，漏掉 91–100。应写 `score >= 90`。
4	语法错误	`elif score >= 60` 末尾缺少英文冒号。
5	缩进错误	`grade = "P"` 没有缩进到对应代码组内。Python 用缩进定义分支体。
6	语法错误	`else score < 60:` 不合法；`else` 后不能写条件，只能写 `else:`。
7	输出格式	最后一行缺少 f-string 前缀，会原样输出 `{grade}`。

序号	类别	错误及后果
8	路径定义	越界路径没有给 grade 赋值，却仍继续访问它，可能产生 NameError。必须保证所有路径都有确定输出。

10.2.2 (2) 完整修正版

下面的写法让每条路径都给 result 赋值，最后统一输出：

```
score = int(input("请输入0~100之间的整数成绩: "))

if score < 0 or score > 100:
    result = "输入无效"
elif score >= 90:
    grade = "A"
    result = f"等级: {grade}"
elif score >= 60:
    grade = "P"
    result = f"等级: {grade}"
else:
    grade = "F"
    result = f"等级: {grade}"

print(result)
```

分支覆盖推导

合法输入的等级函数为

$$G(s)=\begin{cases}\mathrm{A}, & 90\le s\le 100,\\ \mathrm{P}, & 60\le s<90,\\ \mathrm{F}, & 0\le s<60.\end{cases}$$

程序先排除

$$s<0\vee s>100.$$

到达 score >= 90 时，已经知道$s\le 100$，所以该分支自动覆盖$90\le s\le 100$。到达 score >= 60 时，还知道上一条件$s\ge 90$ 为假，因此自动得到$60\le s<90$。最后 else 同时知道$s\ge 0$ 且$s<60$，恰好覆盖 F。

10.2.3 (3) 边界测试

输入	命中的条件路径	输出
-1	score < 0	输入无效
59	前三条件均不成立	等级：F
60	score >= 60	等级：P
89	score >= 60	等级：P
90	score >= 90	等级：A
100	未越界且 score >= 90	等级：A
101	score > 100	输入无效

易错点

越界条件必须用 or：小于 0 或大于 100，任一成立都无效。若误写成 score < 0 and score > 100，同一个数不可能同时满足两项，该分支永远不会成立。题目假设输入文本可转为整数；处理 abc 需要异常处理，超出本组图片的知识范围。

10.3 第 23 题：票种分段程序

10.3.1 (1) 完整程序

```python
age = int(input("请输入整数年龄："))

if age < 0:
    ticket = "输入无效"
elif age <= 5:
    ticket = "免票"
elif age <= 17:
    ticket = "未成年人票"
elif age <= 64:
    ticket = "成人票"
else:
    ticket = "长者票"

print(f"票种：{ticket}")
```

10.3.2 (2) 为什么后续分支只写上界

if--elif--else 从上到下检查，并只执行第一个成立的分支。能到达后续分支，已经隐含前面条件全部为假：

- 到达 age <= 5 时，已知age < 0 为假，即age ≥ 0，合并得到$0 \leq a \leq 5$。

- 到达 `age <= 17` 时，已知$a > 5$。题设限定整数年龄，因此$a \geq 6$，合并得到$6 \leq a \leq 17$。
- 到达 `age <= 64` 时，已知$a > 17$，即整数$a \geq 18$，得到$18 \leq a \leq 64$。
- 到达 `else` 时，已知$a > 64$，即整数$a \geq 65$。

  在整数集合上，五段形成互斥且无遗漏的覆盖：

$$(-\infty, -1] \cup [0, 5] \cup [6, 17] \cup [18, 64] \cup [65, \infty).$$

“互斥”表示一个年龄只属于一段；“完备”表示每个整数年龄都属于其中一段。

10.3.3 (3) 边界输出

年龄	输出	年龄	输出
-1	票种：输入无效	17	票种：未成年人票
0	票种：免票	18	票种：成人票
5	票种：免票	64	票种：成人票
6	票种：未成年人票	65	票种：长者票

10.3.4 (4) 错误条件的后果

若把免票分支写成 `elif age < 5:`，就把闭区间上端点 5 排除。年龄 5 会继续向下，满足 `age <= 17`，被错误判为“未成年人票”。漏判边界值为

$$\boxed{5}.$$

易错点

若把所有 `elif` 改成独立 `if`，一个年龄可能同时满足多个上界条件，`ticket` 会被重复赋值，最终结果由最后一次成立的判断决定，程序语义完全改变。

10.4 第 24 题：公历闰年

10.4.1 (1) 数学命题

$$\boxed{L(y) = \big(400 \mid y\big) \vee \big((4 \mid y) \wedge (100 \nmid y)\big)}$$

使用余数写成

$$\boxed{L(y) = \big(y \bmod 400 = 0\big) \vee \big((y \bmod 4 = 0) \wedge (y \bmod 100 \neq 0)\big)}.$$

符号逐一解释：

- $a \mid b$：b 除以a 的余数为 0。

- $a \nmid b$：余数不为 0。
- $\wedge$：两项都必须成立。
- $\vee$：至少一项成立。

10.4.2 (2) 完整程序

```
year = int(input("请输入年份: "))

if year <= 0:
    print("年份必须为正整数")
elif year % 400 == 0 or (
    year % 4 == 0 and year % 100 != 0
):
    print(f"{year}是闰年")
else:
    print(f"{year}不是闰年")
```

换行处位于圆括号内部，因此是合法的隐式续行。也可以把普通闰年路径写在 `or` 左侧：

```
(year % 4 == 0 and year % 100 != 0) or year % 400 == 0
```

交换 `or` 两侧不改变真值，但可能改变求值顺序；两侧表达式在本题中都无副作用，所以判定结果相同。

10.4.3 (3) 逐年验证

年份	整除关系	代入逻辑	结论
1896	被 4 整除，不被 100 整除	普通闰年路径为 `True and True`	闰年
1900	被 4、100 整除，不被 400 整除	`False or (True and False)`	平年
2000	被 400 整除	第一项直接为 `True`	闰年
2023	不被 4 整除	两条路径都不成立	平年
2024	被 4 整除，不被 100 整除	普通闰年路径为真	闰年
2100	被 100 整除，不被 400 整除	与 1900 同类	平年

对应关键余数：

$$
\begin{aligned}
1896 \bmod 4 = 0, \quad & 1896 \bmod 100 \neq 0, \\
1900 \bmod 100 = 0, \quad & 1900 \bmod 400 \neq 0, \\
2000 \bmod 400 = 0, & \\
2023 \bmod 4 \neq 0, & \\
2024 \bmod 4 = 0, \quad & 2024 \bmod 100 \neq 0, \\
2100 \bmod 100 = 0, \quad & 2100 \bmod 400 \neq 0.
\end{aligned}
$$

10.4.4 (4) 两类典型错误

错误一：只写“能被 4 整除”

```
year % 4 == 0
```

这会错误接纳 1900、2100。世纪年必须进一步检查 400 年例外，所以“每四年一闰”只是普通年份的简化记忆。

错误二：整百年逻辑方向写反

```
year % 100 == 0 or year % 400 == 0
```

因为 400 的倍数必然也是 100 的倍数，上式的第一项已把所有世纪年接纳，导致 1900、2100 被误判。

若采用分层判断，正确顺序之一是：

```
if year % 400 == 0:
    print("闰年")
elif year % 100 == 0:
    print("平年")
elif year % 4 == 0:
    print("闰年")
else:
    print("平年")
```

必须先检查 400；若先把所有 100 的倍数判为平年，2000 会在到达 400 分支之前被错误截获。

10.4.5 400 年周期的独立验算

在任意连续 400 个完整年份构成的公历周期中：

$$
\begin{aligned}
\#(4 \text{ 的倍数}) &= \frac{400}{4} = 100, \\
\#(100 \text{ 的倍数}) &= \frac{400}{100} = 4, \\
\#(400 \text{ 的倍数}) &= \frac{400}{400} = 1.
\end{aligned}
$$

先按每四年一闰得到 100 个，剔除 4 个世纪年，再恢复 1 个 400 倍数，故

$$100 - 4 + 1 = 97$$

个闰年，303 个平年。总天数

$$97 \times 366 + 303 \times 365 = 146097,$$

平均每年

$$\frac{146097}{400} = 365.2425 \text{ 日}.$$

这与美国海军天文台给出的公历规则和平均年长一致。[S11][S12]

第四部分

核验附录

附录 A　答案速查与难度分布

A.1　客观题答案速查

题号	答案	题号	答案	题号	答案
1	C	**6**	B	**11**	A、B、C、E
2	D	**7**	B	**12**	A、C、E
3	B	**8**	C	**13**	A、C、D
4	B	**9**	A	**14**	A、B、E
5	A	**10**	A	**15**	A、C、E

题号	填空	题号	填空
16	`int`	**19**	`elif; else`
17	`f`	**20**	`!=; ==`
18	`n >= -5 and n <= 5`		

A.2　题量与难度审计

题型	基础	中等	提升	合计	题号
单选	4	4	2	10	1–10
多选	1	4	0	5	11–15
填空	2	2	1	5	16–20
综合	0	2	2	4	21–24
总计	7	12	5	24	

难度由单概念识别逐步过渡到边界、继承细节、分支覆盖与程序纠错。基础题保证核心语法不失分；中等题突出执行流程；提升题要求同时处理多个约束。

A.3　知识覆盖

模块	覆盖题号	代表性陷阱
变量与赋值	2、21–24	重新赋值不会让另一名称自动变化；分支路径上的名称必须先绑定

模块	覆盖题号	代表性陷阱
基本类型	1、3、9、11、16	`input()` 返回 `str`；`bool` 是 `int` 子类；非空字符串真值为真
输入输出	1、4、11、16、17、21–24	数值输入需显式转换；f-string需要 `f` 前缀
比较运算	3、6、8、10、12、18–24	`=` 与 `==`；严格与非严格边界；闭区间端点
逻辑运算	5、8、10、12、15、18、20、22、24	优先级、短路、区间内用 `and`、越界用 `or`
条件结构	6、7、13、14、19、21–24	独立 `if` 与分支链；`else` 归属；冒号与缩进
分段与边界	6、8、15、18–24	59/60、89/90、5/6、1900/2000 等关键边界

附录 B　资料来源与全球检索核验

检索与核验时间为 2026-07-25。Python 语义优先采用 Python Software Foundation 维护的当前稳定版 Python 3.14.6 官方文档、语言参考与正式 PEP；闰年规则再以 CPython 源码、美国海军天文台和 NASA 资料交叉核验。网络题库只用于观察常见题型，不复制题干或答案；本册 24 题均重新命制。

B.1　Python 官方语言与标准库资料

[**S1**] Python Language Reference, Objects, values and types. 用于核验对象的标识、类型、值与名称绑定。

[**S2**] Python Language Reference, Assignment statements. 用于核验右侧先求值、名称绑定和赋值目标。

[**S3**] Python Standard Library, Built-in Functions. 具体使用 input(), print(), type() 和 isinstance() 条目。

[**S4**] Python Standard Library, Built-in Types; Truth Value Testing; Boolean Type.

[**S5**] Python Language Reference, Comparisons, Boolean operations 与 Operator precedence.

[**S6**] Python Language Reference, The if statement.

[**S7**] Python Language Reference, Indentation; PEP 8: Indentation.

[**S8**] Python Language Reference, f-strings; Format Specification Mini-Language.

[**S9**] PEP 498: Literal String Interpolation; PEP 701: Syntactic formalization of f-strings.

[**S10**] Python Tutorial, if Statements 与 Input and Output.

B.2　闰年规则的交叉来源

[**S11**] Python Standard Library, calendar.isleap(); CPython `Lib/calendar.py` 源码.

[**S12**] U.S. Naval Observatory, Leap Years 与 Introduction to Calendars.

[**S13**] NASA Goddard Space Flight Center, Calendars and Their History.

B.3　核验结论摘要

- `input()` 返回 `str`，数值语义必须显式转换。

- `bool` 是 `int` 子类，因此 `isinstance(True, int)` 为真。
- `and`/`or` 会短路并返回操作数，`not` 总返回布尔值。
- 链式比较的中间表达式至多求值一次。
- 一条 `if--elif--else` 链只执行第一个成立分支；缩进是语法。
- 公历 400 年周期有 97 个闰年，平均年长 365.2425 日。

附录 C　最终自检清单

检查项	结果
图片重构	四张图均按知识顺序重新编码，并对含混或不精确处单独校正
题量	单选 10、多选 5、填空 5、综合 4，共 24 题
难度	基础 7、中等 12、提升 5；由概念识别过渡到综合程序
典型性	覆盖输入类型、边界、分支配对、继承细节、短路、闰年世纪例外
答案	每题有标准答案；选择题逐项判断；综合题逐句、逐式、逐边界展开
来源	Python 官方文档与正式 PEP 为主，闰年由 CPython、USNO、NASA 交叉核验
版本	按 Python 3 当前稳定文档核验；明确提示 Python 3.12+ 的 f-string 语法更新

完成题目之后，最重要的不是记住某个选项，而是能把“类型、真值、求值顺序、边界、分支覆盖”五件事逐步说清楚。